

HoneyBadger BFT – C++ implementacija

Autor: Filip Stevanović E2 116/2024

Profesor: Darko Čapko

Asistent: Dragan Zarić

Fakultet tehničkih nauka

2026

Sadržaj

1. Uvod
2. Motivacija i problem konsenzusa
3. Osnovni koncept HoneyBadger algoritma
4. Arhitektura sistema
5. Implementacija u C++
6. Zaključak
7. Literatura

1.Uvod

Distribuirani sistemi predstavljaju osnovu modernih informacionih tehnologija, gdje više nezavisnih čvorova sarađuje kako bi izvršili određene zadatke i održali zajedničko stanje sistema. Jedan od ključnih izazova u ovim sistemima jeste postizanje konsenzusa između čvorova.

U realnim uslovima, čvorovi mogu biti nepouzdati, a komunikacija između njih može biti nepredvidljiva. Poseban problem predstavljaju situacije u kojima se pojedini čvorovi ponašaju proizvoljno ili zlonamjerno.

U ovom radu implementiran je pojednostavljeni model **HoneyBadger BFT algoritma** u programskom jeziku C++. Sistem simulira:

- razmjenu poruka između čvorova
- generisanje i distribuciju transakcija
- grupisanje podataka u epohe
- postizanje saglasnosti nad skupom transakcija

Cilj rada je demonstracija rada distribuiranog sistema koji funkcioniše u asinhronom okruženju.

2. Motivacija i problem konsenzusa

U distribuiranim sistemima, više nezavisnih čvorova sarađuje kako bi održali zajedničko stanje sistema i izvršavali operacije nad tim stanjem. Jedan od ključnih izazova u ovakvim sistemima jeste postizanje konsenzusa, odnosno saglasnosti svih ispravnih čvorova o redoslijedu i sadržaju operacija.

U idealnim uslovima pretpostavlja se da svi čvorovi funkcionišu ispravno i da je komunikacija između njih pouzdana. Međutim, u realnim sistemima ova pretpostavka ne važi. Mogu se javiti različiti problemi, kao što su: nepredvidljiva kašnjenja, promjena redoslijeda poruka i zlonamjerni čvorovi. Posebno složen problem predstavljaju situacije u kojima se čvorovi ponašaju proizvoljno, odnosno šalju netačne ili kontradiktorne informacije različitim učesnicima sistema. U takvim uslovima ne postoji jednostavan način da se utvrdi koji čvor govori istinu, što može dovesti do nekonzistentnog stanja sistema.

Da bi distribuirani sistem bio pouzdan, neophodno je da obezbedi sledeće osobine:

- **safety (bezbjednost)** – svi ispravni čvorovi dolaze do istog rezultata
- **liveness (napredak)** – sistem nastavlja sa radom i donosi odluke

HoneyBadger BFT algoritam riješava ovaj problem omogućavajući postizanje konsenzusa u asinhronom okruženju, bez oslanjanja na vremenska ograničenja mreže.

Osnovni uslov za ispravno funkcionisanje sistema jeste da ukupan broj čvorova zadovoljava relaciju:

$$n \geq 3f + 1$$

gde:

- **n** predstavlja ukupan broj čvorova u sistemu

- **f** predstavlja maksimalan broj neispravnih (Byzantine) čvorova

Ovaj uslov garantuje da većina čvorova ostaje ispravna i da sistem može da postigne konsenzus čak i u prisustvu grešaka.

Na taj način, HoneyBadger omogućava stabilan i pouzdan rad distribuiranog sistema u realnim, nepredvidljivim mrežnim uslovima.

3.Osnovni koncept HoneyBadger algoritma

Osnovna ideja algoritma zasniva se na radu epohe.

Epohe - sistem radi u diskretnim koracima koji se nazivaju epohe. Svaka epoha predstavlja jednu rundu razmijene podataka između čvorova. U okviru jedne epohe svaki čvor generiše transakcije, formira batch (grupu transakcija) i šalje ga ostalim čvorovima.

Transakcije - predstavljaju osnovne jedinice podataka koje čvorovi razmjenjuju. Svaka transakcija sadrži: identifikator, identitet čvora koji ju je kreirao i sadržaj (payload)

Proposal (batch) - čvor grupiše više transakcija u jednu strukturu koja se naziva proposal. Proposal sadrži više transakcija koje se šalju svim čvorovima u mreži. Predstavlja kandidat za konačni konsenzus

Razmjena poruka - Komunikacija između čvorova odvija se putem mrežnog sloja koji ne garantuje redoslijed poruka. Mreža može isporučiti poruke sa kašnjenjem kao i promijeniti redoslijed isporuke

Postizanje saglasnosti – svaki čvor prima proposal drugih čvorova, filtrira duplicate i skladišti validne transakcije. Na kraju epohe svi ispravni čvorovi dolaze do istog skupa transakcija

4.Arhitektura sistema

Sistem se sastoji od sljedećih komponenti:

1. **Node** – predstavlja jedan čvor sistema koji je zadužen za generisanje transakcije, kreiranje proposal-a, obradu pristiglih poruka i čuvanje lokalnog stanja.
2. **Network** – simulira mrežu koja šalje poruke između čvorova. Koristi queue model (red) za slanje poruka i podržava broadcast.
3. **Transaction** – struktura koja sadrži ID transakcije, ID čvora koji ju je kreirao i sadržaj.
4. **Proposal** – sadrži skup transakcija koje jedan čvor predlaže ostalim čvorovima.

5. Implementacija u C++

U ovoj sekciji prikazana je implementacija pojednostavljenog **HoneyBadger BFT** modela u programskom jeziku C++. Sistem je organizovan kroz više klasa koje predstavljaju osnovne komponente distribuiranog okruženja. Najvažnije među njima su: **Node**, **Network**, **Transaction** i **Proposal**.

5.1 Inicijalizacija sistema

Na početku programa kreira se mreža i skup čvorova. Svaki čvor dobija svoj identifikator, povezuje se sa mrežom i registruje u sistemu. Takođe se definišu osnovni parametri simulacije: broj čvorova, broj epoha i maksimalan broj transakcija koje jedan proposal može da sadrži.

```
constexpr int totalNodes = 4;
constexpr int totalEpochs = 4;
constexpr int maxTransactionsPerProposal = 2;

Network network;
std::vector<std::unique_ptr<Node>> nodes;

for (int i = 0; i < totalNodes; ++i)
{
    nodes.push_back(std::make_unique<Node>(i));
    nodes.back()->AttachNetwork(&network);
    nodes.back()->SetTotalNodes(totalNodes);
    network.RegisterNode(nodes.back().get());
}
```

U ovom dijelu programa vidi se da se svi čvorovi nalaze u istoj simuliranoj mreži. Svaki objekat klase **Node** predstavlja jednu repliku sistema. Pozivom **AttachNetwork()** čvor dobija referencu na mrežni sloj, dok se metodom **RegisterNode()** taj čvor dodaje u internu strukturu mreže kako bi kasnije mogao da prima poruke.

5.2 Generisanje transakcija

Osnovna jedinica podataka u sistemu je transakcija. Svaki čvor može lokalno da generiše novu transakciju, a zatim da je pošalje ostalim čvorovima u mreži.

```
Transaction tx0a = nodes[0]->GenerateTransaction("tx from node 0 - A");
nodes[0]->BroadcastTransaction(tx0a, totalNodes);
Transaction tx0b = nodes[0]->GenerateTransaction("tx from node 0 - B");
nodes[0]->BroadcastTransaction(tx0b, totalNodes);

Transaction tx1a = nodes[1]->GenerateTransaction("tx from node 1 - A");
nodes[1]->BroadcastTransaction(tx1a, totalNodes);
```

```
Transaction tx2a = nodes[2]->GenerateTransaction("tx from node 2 - A");
nodes[2]->BroadcastTransaction(tx2a, totalNodes);

Transaction tx3a = nodes[3]->GenerateTransaction("tx from node 3 - A");
nodes[3]->BroadcastTransaction(tx3a, totalNodes);
Transaction tx3b = nodes[3]->GenerateTransaction("tx from node 3 - B");
nodes[3]->BroadcastTransaction(tx3b, totalNodes);
```

5.3 Serijalizacija i deserijalizacija podataka

Pošto se poruke u simulaciji prenose kroz mrežu kao tekstualni sadržaj, transakcije je potrebno pretvoriti u string. Za to se koristi serijalizacija.

```
std::string Node::SerializeTransaction(const Transaction& tx) const
{
    return std::to_string(tx.id) + "|" +
           std::to_string(tx.creatorNodeId) + "|" +
           tx.payload;
}
```

U ovoj funkciji vrijednosti polja id, creatorNodeId i payload spajaju se u jedan string korišćenjem separatora "|". Takav string je jednostavan za slanje kroz mrežu. Kada čvor primi poruku, potrebno je izvršiti obrnuti postupak, odnosno deserijalizaciju.

```
Transaction Node::DeserializeTransaction(const std::string& data) const
{
    const std::size_t firstSep = data.find('|');
    const std::size_t secondSep = data.find('|', firstSep + 1);

    if (firstSep == std::string::npos || secondSep == std::string::npos)
    {
        throw std::runtime_error("Invalid transaction format");
    }

    Transaction tx;
    tx.id = std::stoi(data.substr(0, firstSep));
    tx.creatorNodeId = std::stoi(data.substr(firstSep + 1, secondSep -
firstSep - 1));
    tx.payload = data.substr(secondSep + 1);

    return tx;
}
```

5.4. Lokalni pool transakcija

Svaki čvor posjeduje lokalni skup transakcija, odnosno transaction pool. U njemu se čuvaju: transakcije koje je čvor sam kreirao i transakcije koje je primio od drugih čvorova. Prilikom obrade novih poruka, čvor provjerava da li je transakciju već ranije primio. Ako jeste, takva transakcija se ignoriše.

```

if (!HasTransaction(tx.id))
{
    AddTransaction(tx);
    std::cout << " -> added to local pool\n";
}
else
{
    std::cout << " -> duplicate ignored\n";
}

```

5.5 Obrada inbox-a

Kada mreža isporuči poruku nekom čvoru, ta poruka se najprije smješta u njegov inbox. Sama obrada se vrši kasnije, pozivom metode **ProcessInbox()**.

```

void Node::ProcessInbox()
{
    std::cout << "Node " << mId << " processing inbox:\n";

    if (mInbox.empty())
    {
        std::cout << " [empty]\n";
        return;
    }

    for (const auto& msg : mInbox)
    {
        if (msg.type == MessageType::TransactionBroadcast)
        {
            Transaction tx = DeserializeTransaction(msg.payload);

            std::cout
                << " Received transaction from Node " << msg.senderId
                << " | tx.id=" << tx.id
                << " | creator=" << tx.creatorNodeId
                << " | payload=\"" << tx.payload << "\"\n";

            if (!HasTransaction(tx.id))
            {
                AddTransaction(tx);
                std::cout << " -> added to local pool\n";
            }
            else
            {
                std::cout << " -> duplicate ignored\n";
            }
        }
    }

    mInbox.clear();
}

```

U ovoj funkciji čvor prolazi kroz sve primljene poruke i na osnovu njihovog tipa odlučuje kako će ih obraditi. Ako je u pitanju poruka tipa **TransactionBroadcast**, vrši se:

- deserijalizacija poruke
- ispis informacija o transakciji
- provjera da li transakcija već postoji
- eventualno dodavanje u lokalni pool

Na kraju obrade inbox se prazni, jer su sve poruke iz te runde već obrađene.

5.6 Kreiranje proposal-a

Kada čvor prikupi dovoljan broj transakcija, može da formira proposal. Proposal predstavlja batch transakcija koje taj čvor predlaže u okviru jedne epohe.

```
Proposal Node::CreateProposal(int maxTransactions) const
{
    Proposal proposal;
    proposal.proposerId = mId;

    const int txCount = std::min<int>(maxTransactions,
static_cast<int>(mTransactionPool.size()));
    for (int i = 0; i < txCount; ++i)
    {
        proposal.transactions.push_back(mTransactionPool[i]);
    }

    return proposal;
}
```

Ova funkcija prolazi kroz lokalni pool transakcija i uzima najviše **maxTransactions** elemenata. Time se implementira batching, odnosno grupisanje više transakcija u jednu poruku višeg nivoa.

5.7. Serijalizacija proposal-a

Analogno transakcijama, vrši se serijalizacija i deserijalizacija proposal-a.

```
std::string Node::SerializeProposal(const Proposal& proposal) const
{
    std::ostringstream oss;
    oss << proposal.proposerId;

    for (const auto& tx : proposal.transactions)
    {
        oss << '\n' << SerializeTransaction(tx);
    }

    return oss.str();
}
```

```

Proposal Node::DeserializeProposal(const std::string& data) const
{
    std::istringstream iss(data);
    std::string line;
    Proposal proposal{};

    if (!std::getline(iss, line))
    {
        throw std::runtime_error("Invalid proposal payload format.");
    }

    proposal.proposerId = std::stoi(line);

    while (std::getline(iss, line))
    {
        if (!line.empty())
        {
            proposal.transactions.push_back(DeserializeTransaction(line));
        }
    }

    return proposal;
}

```

5.8. Broadcast proposal-a

Nakon formiranja proposal-a, čvor ga šalje svim ostalim čvorovima u sistemu.

```

void Node::BroadcastProposalBatch(const Proposal& proposal, int totalNodes)
{
    if (mBehavior == ByzantineBehavior::SilentProposal)
    {
        std::cout << "Node " << mId << " is Byzantine: stays silent and sends no proposal\n";
        return;
    }

    Proposal outgoingProposal = proposal;

    if (mBehavior == ByzantineBehavior::InvalidProposal)
    {
        outgoingProposal = BuildInvalidProposalFrom(proposal);
        std::cout << "Node " << mId << " is Byzantine: broadcasting INVALID proposal\n";
        BroadcastProposalHonest(outgoingProposal, totalNodes);
        return;
    }

    if (mBehavior == ByzantineBehavior::EquivocatingProposal)
    {
        std::cout << "Node " << mId << " is Byzantine: broadcasting DIFFERENT proposals to different nodes\n";
        BroadcastProposalEquivocating(proposal, totalNodes);
        return;
    }
}

```



```

std::cout << "Node " << mId << " broadcasts proposal batch\n";
BroadcastProposalHonest(outgoingProposal, totalNodes);
BroadcastEchoForProposal(outgoingProposal, totalNodes);
}

```

Funkcija `BroadcastProposalBatch` zadužena je za slanje `proposal`-a ostalim čvorovima u mreži, pri čemu uvažava trenutno ponašanje čvora. U zavisnosti od toga da li je čvor pošten ili Byzantine, funkcija određuje način slanja:

- ako je čvor **Silent**, ne šalje ništa;
- ako je **Invalid**, šalje izmijenjen (neispravan) `proposal`;
- ako je **Equivocating**, šalje različite `proposal`-e različitim čvorovima;
- ako je čvor **pošten**, šalje validan `proposal` svim čvorovima i dodatno propagira informaciju putem echo mehanizma.

Na ovaj način funkcija omogućava simulaciju različitih tipova ponašanja čvorova u distribuiranom sistemu.

5.9. Obrada `proposal` poruka

Kada `proposal` stigne do nekog čvora, on se obrađuje u okviru `inbox`-a na isti način kao i ostale poruke.

```

if (msg.type == MessageType::ProposalBroadcast)
{
    Proposal proposal = DeserializeProposal(msg.payload);

    std::cout
        << " Received PROPOSAL from Node " << msg.senderId
        << " | proposer=" << proposal.proposerId
        << " | tx_count=" << proposal.transactions.size();

    mDirectProposalsBySender[msg.senderId] = proposal;
}

```

U ovom koraku čvor:

- deserijalizuje `proposal`
- čita koji čvor ga je poslao
- proverava koliko transakcija sadrži
- čuva ga u lokalnoj listi primljenih `proposal`-a

Na kraju svake epohe svaki ispravan čvor raspolaže skupom `proposal`-a pristiglih od drugih učesnika sistema.

5.10. Mrežni sloj i asinhronost

Klasa **Network** simulira komunikaciju između čvorova. Umesto direktnog slanja poruka, sve poruke se najpre smeštaju u red čekanja, a zatim se kasnije isporučuju.

Najvažnija funkcija mreže je **DeliverAll()**:

```

void Network::DeliverAll()
{
    std::cout << "\nNetwork delivering messages...\n";

    std::random_device rd;
    std::mt19937 g(rd());

    std::shuffle(mPendingMessages.begin(), mPendingMessages.end(), g);

    for (const auto& msg : mPendingMessages)
    {
        auto it = mNodes.find(msg.receiverId);
        if (it != mNodes.end() && it->second != nullptr)
        {
            it->second->ReceiveMessage(msg);
        }
    }

    mPendingMessages.clear();
}

```

Ova funkcija prvo nasumično miješa redoslijed poruka pomoću `std::shuffle`, a zatim ih isporučuje odgovarajućim čvorovima. Time se simulira asinhrona mreža u kojoj:

- poruke ne stižu redom kojim su poslate
- može postojati kašnjenje
- čvorovi ne mogu unaprijed znati tačan redoslijed događaja

Ovaj aspekt je veoma važan, jer **HoneyBadger BFT** upravo i jeste namijenjen radu u uslovima nepredvidive mreže.

5.11. Simulacija neispravnih čvorova

U implementaciji je moguće simulirati i čvorove koji se ne ponašaju korektno.

```

enum class ByzantineBehavior
{
    Honest,
    SilentProposal,
    InvalidProposal,
    EquivocatingProposal
};

```

- **Honest** – čvor poštuje protokol i šalje validne podatke
- **SilentProposal** – čvor ne učestvuje u slanju proposal-a
- **InvalidProposal** – čvor šalje netačne ili oštećene podatke
- **EquivocatingProposal** – čvor šalje različite informacije različitim čvorovima

Na ovaj način se može testirati otpornost sistema na nepravilno ponašanje pojedinih učesnika. Ideja je da ispravni čvorovi i dalje nastave da rade korektno, pod uslovom da ukupan broj čvorova zadovoljava uslov: $n \geq 3f + 1$.

6.Zaključak

Implementacija HoneyBadger BFT modela u ovom projektu potvrđuje osnovne principe distribuiranog konsenzusa u asinhronom okruženju. Sistem uspješno demonstrira razmenu transakcija između čvorova, formiranje batch-eva i postizanje zajedničkog rezultata uprkos nepredvidivom redoslijedu poruka u mreži.

Ključni rezultati implementacije su:

- uspješna simulacija rada po epohama i razmene proposal-a između čvorova;
- formiranje batch-eva transakcija i njihova distribucija kroz mrežu;
- očuvanje konzistentnosti lokalnog stanja kod svih ispravnih čvorova;
- otpornost sistema na nepredvidiv redoslijed isporuke poruka;
- simulacija neispravnih (Byzantine) čvorova i analiza njihovog uticaja kroz logove.

Dobijeni rezultati pokazuju da svi ispravni čvorovi, nakon razmene poruka, dolaze do istog skupa transakcija, čime se postiže osnovni cilj distribuiranog konsenzusa.

7.Reference

- Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, Dawn Song – *“The Honey Badger of BFT Protocols”*, 2016.